



Πανεπιστήμιο Πατρών
Τμήμα Ηλεκτρολόγων Μηχανικών και Τεχνολογίας Υπολογιστών

Προηγμένες Τεχνικές Προγραμματισμού

Αναφορά Εργασίας Εξαμήνου

Ομάδα 09

Ζώντος Μιχαήλ 1097318

Οικονόμου Ιωάννης 1092684

Ημ/νία παράδοσης: 01/06/2026

1. Εισαγωγή

Η διαχείριση πόρων σε σύγχρονα έξυπνα περιβάλλοντα, απαιτεί τη μετάβαση από στατικά, προκαθορισμένα μοντέλα λειτουργίας σε δυναμικά συστήματα (edge-device systems) που βασίζονται σε δεδομένα πραγματικού. Ένα από τα χαρακτηριστικά προβλήματα υποδομών σε γενικούς χώρους αποτελεί η διαχείριση των απορριμμάτων. Τα παραδοσιακά συστήματα συλλογής στηρίζονται σε στατικά δρομολόγια μέσω στατικών μοντέλων, γεγονός που οδηγεί συχνά είτε σε υπερχειλίση των κάδων λόγω απρόβλεπτης αυξημένης κίνησης (π.χ. σε ώρες αιχμής στα εργαστήρια), είτε σε άσκοπους ελέγχους άδειων κάδων, σπαταλώντας πόρους.

Για την επίλυση αυτού του προβλήματος, στο πλαίσιο του μαθήματος «Προηγμένες Τεχνικές Προγραμματισμού», σχεδιάστηκε και αναπτύχθηκε το σύστημα SmartBin. Πρόκειται για ένα event-driven σύστημα IoT, το οποίο υιοθετεί τους συμβατικούς κάδους ως smart-nodes που παρακολουθεί συνεχώς τη χρήση τους και την πληρότητά τους.

Κύριοι Στόχοι

Ο πρωταρχικός στόχος του έργου είναι η δημιουργία μιας επεκτάσιμης αρχιτεκτονικής που συλλέγει, αναλύει και διαθέτει σημασιολογικά δεδομένα τηλεμετρίας. Ειδικότερα, το σύστημα σχεδιάστηκε για να επιτύχει τα εξής:

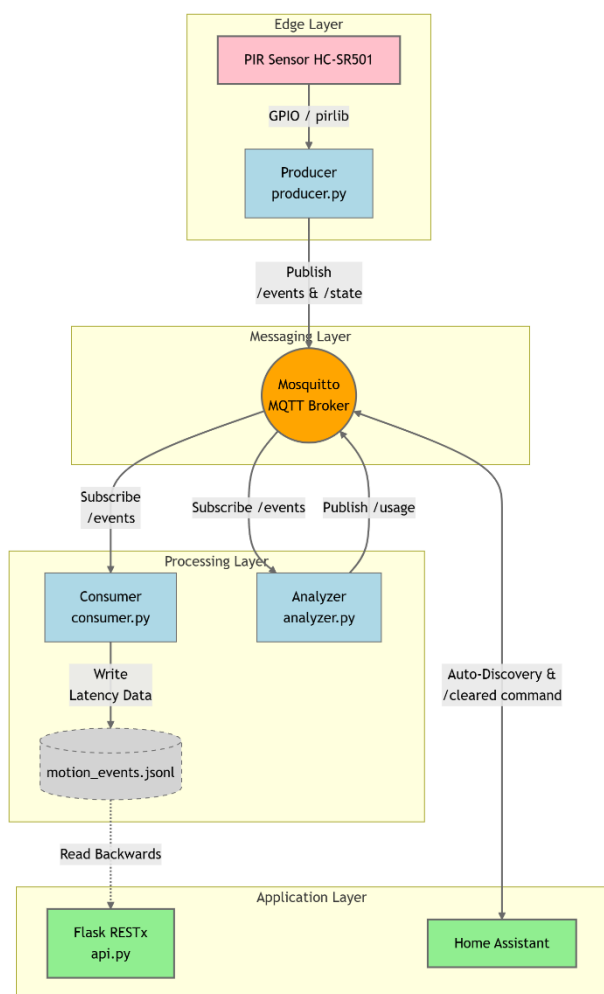
- **Edge Computing:** Χρήση αισθητήρα υπερύθρων (PIR Motion Sensor), ενσωματωμένου σε μικροελεγκτή (Raspberry Pi 5). Η συσκευή αυτή αναλαμβάνει την τοπική επεξεργασία του σήματος (motion events) και την καταγραφή των γεγονότων κίνησης όταν απορρίπτεται κάποιο αντικείμενο στον κάδο.
- **Event-Driven Architecture:** Δημιουργία pipeline που αξιοποιεί το πρωτόκολλο MQTT (μέσω του Mosquitto broker). Υλοποιείται το δίπολο Publisher/Subscriber, επιτρέποντας τον διαχωρισμό της συλλογής δεδομένων από την επεξεργασία τους. Ο παραγωγός (Producer) μεταδίδει raw events, ενώ ο καταναλωτής (Consumer και Analyzer) εγγράφονται στα αντίστοιχα topics για να αναλύσουν τα δεδομένα σε πραγματικό χρόνο.
- **Semantic Models:** Μια από τις σημαντικότερες πτυχές της εργασίας είναι η υιοθέτηση προτύπων του Semantic Web. Τα δεδομένα δεν είναι απλά αδόμητα JSON, αλλά ακολουθούν το πρότυπο JSON-LD, καθιστώντας τα δεδομένα διαλειτουργικά και machine-readable από τρίτα συστήματα.
- **Use of Docker Containers:** Όλα τα υποσυστήματα του λογισμικού, συμπεριλαμβανομένων του Analyzers, του Consumer και του Producer, είναι πλήρως κοντεϊνερποιημένα μέσω Docker. Αυτό εξασφαλίζει την εύκολη ανάπτυξη, εξέλιξη και επεκτασιμότητα του συστήματος.
- **Central Home Assistant Hub:** Η οπτικοποίηση και ο έλεγχος των καταστάσεων των edge nodes υλοποιούνται μέσω του πρωτοκόλλου MQTT Auto-Discovery, επιτρέποντας την άμεση και δυναμική ενσωμάτωση του κάθε κάδου στο περιβάλλον του Home Assistant. Μέσα από αυτό, ο διαχειριστής της εγκατάστασης μπορεί να βλέπει τη χωρητικότητα, την κατάσταση λειτουργίας του αισθητήρα και να στέλνει εντολές "αδειάσματος" (empty bin) που επαναφέρουν την ακολουθία καταμέτρησης στο μηδέν, υλοποιώντας έτσι ένα εικονικό περιβάλλον διαχείρισης κάδων.

2. Προσέγγιση και Αρχιτεκτονική

Η αρχιτεκτονική του συστήματος SmartBin σχεδιάστηκε με γνώμονα την επεκτασιμότητα (scalability). Υιοθετήθηκε ένα κατανεμημένο, event-driven μοντέλο μικροπηρεσιών, το οποίο επιτρέπει την ασύγχρονη επικοινωνία μεταξύ του υλικού στο άκρο (edge hardware) και των εφαρμογών του τελικού χρήστη.

Όπως απεικονίζεται στο **Σχήμα 1**, η τοπολογία του συστήματος οργανώνεται σε τέσσερα διακριτά επίπεδα (layers) συνεχούς ροής δεδομένων:

1. Η παραγωγή της πληροφορίας ξεκινά στο **Edge Layer**, όπου το hardware καταγράφει τα φυσικά γεγονότα και τα μετατρέπει σε ψηφιακά σήματα.
2. Τα δεδομένα προωθούνται στο **Messaging Layer**, το οποίο λειτουργεί ως το κύριο pipeline, διασφαλίζοντας την αξιόπιστη και ασύγχρονη δρομολόγηση των μηνυμάτων.
3. Οι αποσυνδεδεμένες μικροπηρεσίες του **Processing Layer** αναλαμβάνουν την αποθήκευση, τον υπολογισμό μετρικών (όπως το latency) και την εξαγωγή συμπερασμάτων σε πραγματικό χρόνο (virtual sensors).
4. Τέλος, η διαδραστικότητα επιτυγχάνεται στο **Application Layer**, το οποίο καταναλώνει την επεξεργασμένη πληροφορία προσφέροντας τυποποιημένες REST διεπαφές και γραφικά περιβάλλοντα (dashboards) για τη λήψη αποφάσεων.



Σχήμα 1: Αρχιτεκτονική Ροής Δεδομένων του SmartBin. Διακρίνονται τα τέσσερα επίπεδα του συστήματος. Η συλλογή του φυσικού σήματος γίνεται στο Edge Layer (PIR, producer.py), η δρομολόγηση μέσω του Messaging Layer (Mosquitto MQTT Broker), η ασύγχρονη επεξεργασία στο Processing Layer (όπου ο consumer.py καταγράφει τα γεγονότα και ο analyzer.py υπολογίζει το επίπεδο χρήσης) και η τελική διάθεση των δεδομένων στο Application Layer μέσω του Flask API και του Home Assistant.

2.1 Edge/Hardware Layer

Στο Edge Layer, η συλλογή των δεδομένων πραγματοποιείται από αισθητήρες υπερύθρων (PIR HC-SR501) που συνδέονται στα GPIO pins του Raspberry Pi 5, αξιοποιώντας την προσαρμοσμένη βιβλιοθήκη pirlib.

Η ανάγνωση των ψηφιακών σημάτων επιτυγχάνεται μέσω της κλάσης PirSampler, η οποία αξιοποιεί την κλάση DigitalInputDevice της βιβλιοθήκης gpiozero. Ένα από τα βασικά πλεονεκτήματα της αρχιτεκτονικής είναι η υποστήριξη διττής λειτουργίας (dual mode). Πιο συγκεκριμένα, υλοποιήθηκε η κλάση VirtualPirSampler, η οποία επιτρέπει τη λειτουργία του παραγωγού (producer.py) σε περιβάλλον προσομοίωσης εικονικών κάδων (επιλογή –virtual στο docker). Η εικονική γεννήτρια δεδομένων αξιοποιεί στοχαστικές κατανομές και επιτρέπει την ανάπτυξη (scalability) του συστήματος χωρίς την απαίτηση φυσικού εξοπλισμού.

2.2 Event-Driven Architecture & MQTT

Η επικοινωνία του συστήματος βασίζεται στο πρωτόκολλο MQTT, κάνοντας χρήση του Mosquitto Broker. Αντί για άμεσες συνδέσεις, υιοθετείται το δίπολο Publisher/Subscriber, εξασφαλίζοντας το decoupling της παραγωγής δεδομένων από την ανάλυση και την αποθήκευσή τους.

Για τη διασφάλιση της ορθότητας της επικοινωνίας, η αρχιτεκτονική των μηνυμάτων σχεδιάστηκε με τη λογική "API-First". Η δομή των καναλιών (topics) και τα payloads τεκμηριώνονται αυστηρά μέσω του προτύπου AsyncAPI (asyncapi.yaml). Βάσει αυτού του συμβολαίου, το σύστημα επικοινωνεί μέσω της παρακάτω ιεραρχίας θεμάτων (smartbin/{bin_id}/{device_id}/...):

- **/events:** Το βασικό κανάλι τηλεμετρίας. Ο Producer δημοσιεύει τα motion events (σε μορφή JSON-LD), τα οποία καταναλώνονται ασύγχρονα από τον Consumer και τον Analyzer.
- **/usage:** Το κανάλι αναλυτικών δεδομένων. Ο Analyzer δημοσιεύει εδώ το επίπεδο χρήσης του κάδου (idle, low, medium, high).
- **/availability & /coordinates:** Το κανάλι /availability εξυπηρετεί χρήσιμο ρόλο, ενημερώνοντας το δίκτυο αν ο αισθητήρας βγει offline. Παράλληλα, το /coordinates διατηρεί (μέσω MQTT Retain) το τρέχον γεωγραφικό στίγμα του κάδου, επιτρέποντας τη δυναμική χαρτογράφησης του (Home Assistant Dashboard).
- **/seq & /cleared:** Κανάλια ελέγχου χωρητικότητας. Το /seq διατηρεί τον τρέχοντα αύξοντα αριθμό ρίψεων. Όταν ο κάδος αδειάζει (μέσω εντολής στο /cleared), ο Producer μηδενίζει το sequence και συγχρονίζει εκ νέου την κατάσταση.

Το παρακάτω απόσπασμα του AsyncAPI αποτυπώνει την τυποποίηση του καναλιού τηλεμετρίας και του καναλιού ελέγχου διαθεσιμότητας:

<pre>channels: smartbin/{bin_id}/{device_id}/events: description: Stream of raw JSON-LD motion events. subscribe: operationId: subscribeToMotionEvents message: \$ref: '#/components/messages/MotionEvent'</pre>	<pre>smartbin/{bin_id}/{device_id}/availability: description: Last Will and Testament (LWT) status. subscribe: operationId: subscribeToAvailability message: \$ref: '#/components/messages/AvailabilityMessage'</pre>
--	--

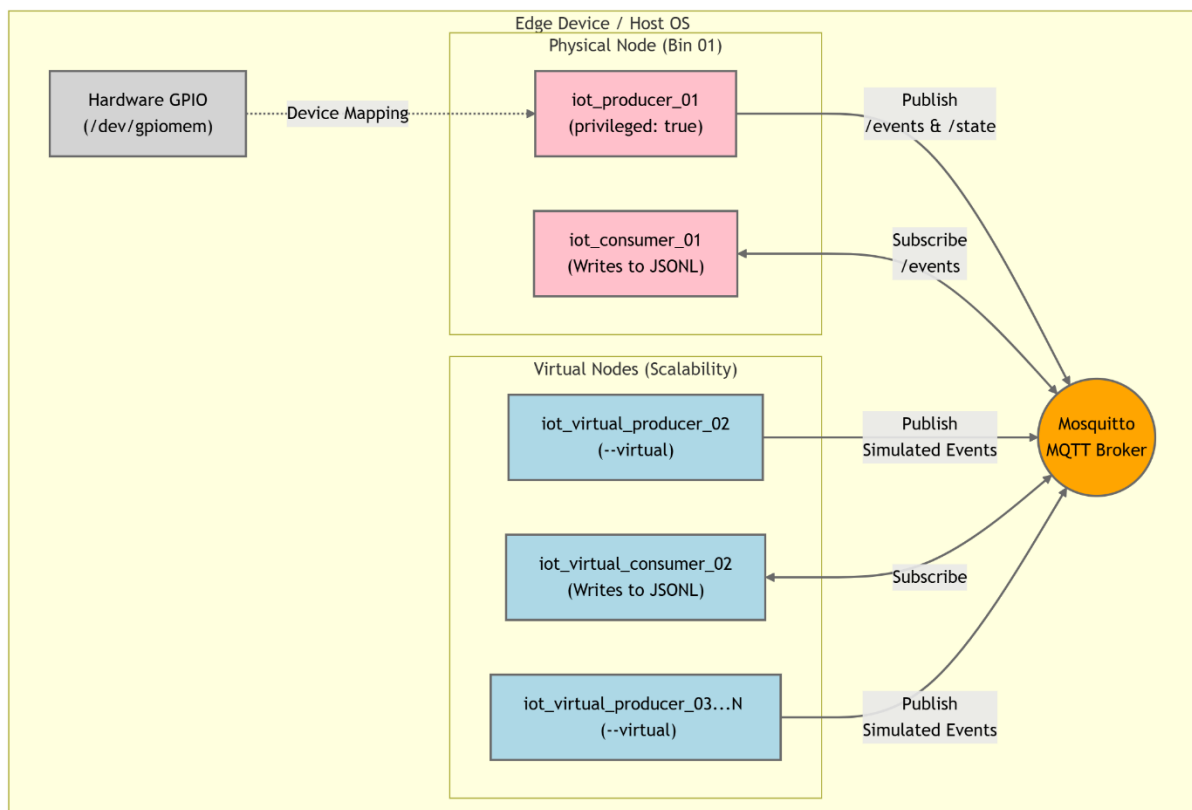
2.3 Containerization, Docker & Scalability

Για την εξασφάλιση της απομόνωσης των dependencies και την υποστήριξη επεκτασιμότητας, ολόκληρη η στοίβα λογισμικού σχεδιάστηκε ως ένα σύνολο ανεξάρτητων μικρουπηρεσιών. Η ανάπτυξη υλοποιείται μέσω του docker-compose.yml.

Κάθε υπηρεσία του συστήματος (Producer, Consumer, Analyzer) εκτελείται στο δικό της αυτόνομο Docker container, επικοινωνώντας άμεσα με τον τοπικό MQTT Broker του Host συστήματος, κάνοντας χρήση της παραμέτρου network_mode: "host".

Το σημαντικότερο πλεονέκτημα αυτής της προσέγγισης είναι η ικανότητα ελέγχου επεκτασιμότητας. Το σύστημα διαχωρίζει πλήρως τους φυσικούς κόμβους από τους εικονικούς κόμβους (virtual nodes), επιτρέποντας την προσομοίωση ενός τεράστιου στόλου από έξυπνους κάδους πάνω στο ίδιο μηχάνημα:

1. **Φυσικός Κόμβος:** Για την πρόσβαση στο πραγματικό hardware (Raspberry Pi), το container του φυσικού κάδου εκτελείται σε λειτουργία υψηλών προνομίων (privileged: true) και κάνει mount τις συσκευές /dev/gpiomem0 και /dev/gpiochip0.
2. **Εικονικοί Κόμβοι:** Για τη δοκιμή του φόρτου (stress) της αρχιτεκτονικής, το docker-compose.yml αναπτύσσει επιπλέον εικονικούς παραγωγούς (virtual_producer_02, virtual_producer_03 κ.ο.κ.). Αυτοί εκτελούνται με την παράμετρο --virtual, παράγοντας στοχαστικά δεδομένα κίνησης χωρίς καμία ανάγκη για πραγματικό hardware και χωρίς αυξημένα δικαιώματα (privileges).



Η ευκολία με την οποία επιτυγχάνεται αυτό το scaling, αποτυπώνεται στον παρακάτω κώδικα διαμόρφωσης, όπου φαίνεται η διαφορά προνομίων μεταξύ ενός φυσικού και ενός εικονικού παραγωγού:

```
# Απόσπασμα από docker-compose.yml: Φυσικός vs Εικονικός Παραγωγός

services:

  producer01:

    container_name: iot_producer_01

    command: python producer.py --broker localhost --topic "smartbin" --device-id pir-01 --bin-id bin-01 --pin 17

    network_mode: "host"

    privileged: true # Υποχρεωτικό για πρόσβαση στα GPIO pins

    devices:

      - "/dev/gpiomem0:/dev/gpiomem"

      - "/dev/gpiochip0:/dev/gpiochip0"

  virtual_producer_02:

    container_name: iot_virtual_producer_02

    # Χρήση της παραμέτρου --virtual για προσομοίωση κλιμάκωσης δικτύου

    command: python producer.py --broker localhost --topic "smartbin" --device-id pir-02 --bin-id bin-02 --pin 0 --virtual

    network_mode: "host"

    # Δεν απαιτούνται privileged δικαιώματα ούτε devices
```

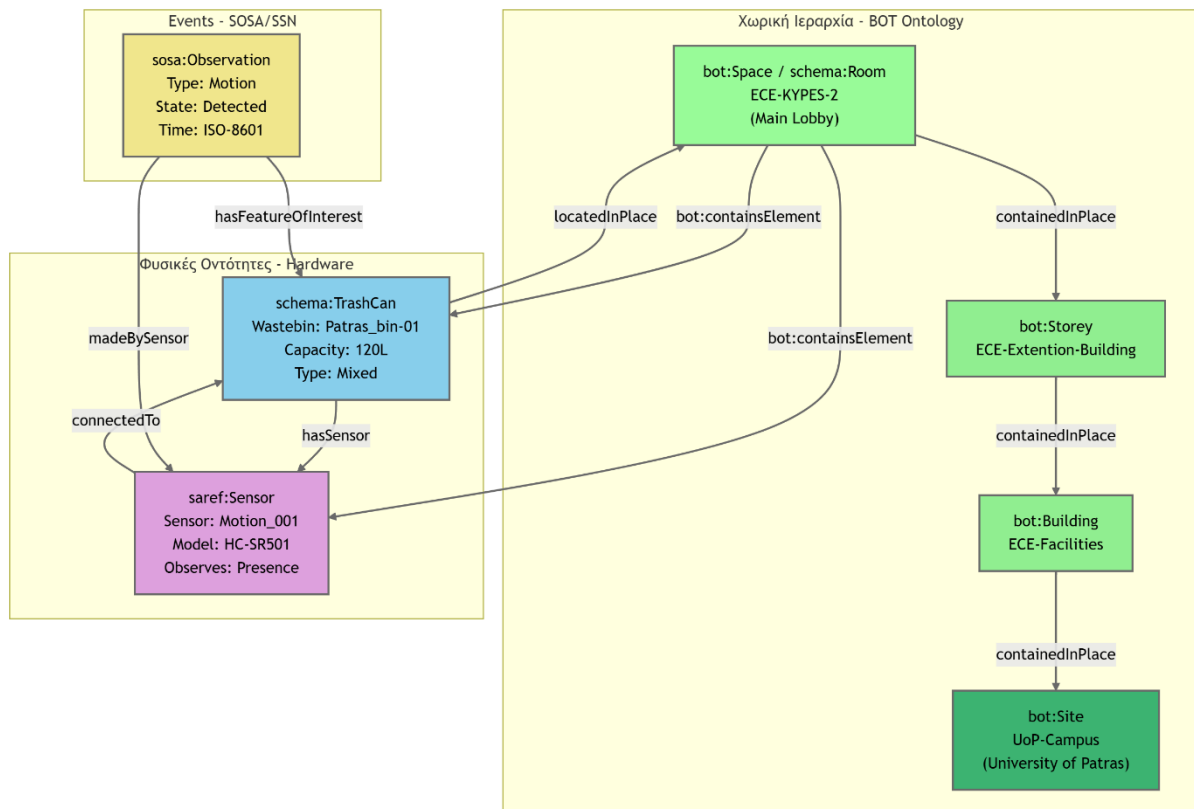
2.4 Semantic Data Modeling (Linked Data/JSON-LD)

Σε επίπεδο δεδομένων, το σύστημα υιοθετεί τεχνολογίες Semantic Web. Όλα τα δεδομένα που διακινούνται και αποθηκεύονται ακολουθούν το πρότυπο JSON-LD. Αυτό μετατρέπει τις απλές συσκευές σε «έξυπνους κόμβους», καθιστώντας τα δεδομένα machine-readable και διαλειτουργικά με τρίτα συστήματα.

Για να επιτευχθεί αυτό, δημιουργήθηκε ένα κεντρικό λεξικό (context.jsonld), το οποίο χαρτογραφεί τις εσωτερικές μεταβλητές του συστήματος πάνω σε καθιερωμένες παγκόσμιες οντολογίες:

1. **Schema.org (schema):** Χρησιμοποιείται για την περιγραφή γενικών ιδιοτήτων, όπως γεωγραφικές συντεταγμένες (GeoCoordinates) και φυσικά αντικείμενα (TrashCan).
2. **SAREF (saref):** Χρησιμοποιείται για την τυποποίηση του εξοπλισμού, κατηγοριοποιώντας τον αισθητήρα κίνησης HC-SR501 ως saref:Sensor.
3. **SOSA/SSN (iot / sosa):** Τα events που παράγει ο producer είναι σημασιολογικά observations.
4. **BOT - Building Topology Ontology (bot):** Χωρική αναπαράσταση του συστήματος, ορίζει το πώς τα αντικείμενα τοποθετούνται μέσα στα κτήρια.

Αντίστοιχα, το μοντέλο του ίδιου του κάδου (wastebin.jsonld) φέρει πλήρη τεχνικά χαρακτηριστικά (όπως χωρητικότητα, υλικό κατασκευής και τύπο απορριμμάτων).



Σχήμα 3: Το Πλήρες Σημασιολογικό Γράφημα του συστήματος SmartBin. Διακρίνονται τρία επίπεδα μοντελοποίησης: Η τοπολογική ιεραρχία του κτηρίου (βάσει της οντολογίας BOT). Οι οντότητες υλικού (κάδος και αισθητήρας) που περιγράφονται βάσει SAREF και Schema.org, και τα δυναμικά γεγονότα τηλεμετρίας (SOSA Observations).

2.5 Application Layer: RESTful API

Για τη διάθεση των δεδομένων σε εξωτερικές εφαρμογές και υπηρεσίες (third-party integrations), αναπτύχθηκε ένα σύγχρονο RESTful API. Η υλοποίηση βασίστηκε στο framework Flask, κάνοντας χρήση της βιβλιοθήκης Flask-RESTx, η οποία προσφέρει παραγωγή και τεκμηρίωση του API μέσω διεπαφής **Swagger UI**.

Σε επίπεδο ανάγνωσης, τα endpoints `/bins` και `/sensors` παρέχουν άμεση πρόσβαση στη χωρική ιεραρχία, ενώ το endpoint `/events` προελαύνει το ιστορικό χρήσης για τη δραστική βελτιστοποίηση της μνήμης. Επιπρόσθετα, η υπηρεσία λειτουργεί ως ενεργός MQTT Client παρασκηνίου, επιτρέποντας στους clients να ανακτούν την τρέχουσα κατάσταση του συστήματος μέσω του `/availability` και το δυναμικό γεωγραφικό στίγμα μέσω του `/coordinates`. Πέραν της συλλογής δεδομένων, το API υποστηρίζει δυνατότητες αμφίδρομου ελέγχου. Η πλήρης ιεραρχία των ομαδοποιημένων μεθόδων παρουσιάζεται στα παρακάτω στιγμιότυπα.

SmartBin API1.0

[Base URL: /]
/swagger.json

REST API for querying Smart Wastebin sensor data, bin status, and live MQTT states.

binsWastebin operations & live data

GET/binsList all registered bins (Static)

GET/bins/{bin_id}Get static details for a specific bin

GET/bins/{bin_id}/availabilityGet the live availability (LWT) of sensors on a specific bin

GET/bins/{bin_id}/coordinatesGet the live GPS coordinates of sensors on a specific bin

POST/bins/{bin_id}/emptiedRecord that a bin was emptied and publish an MQTT update

GET/bins/{bin_id}/eventsGet recent motion events for a specific bin

GET/bins/{bin_id}/sensorsList sensors mounted on a specific bin

sensorsSensor static details

GET/sensorsList all registered sensors (Static)

GET/sensors/{sensor_id}Get static details for a specific sensor

mqttRaw MQTT Topic State Tracking

GET/mqtt/topicsList all known MQTT topics and their last received message

GET/mqtt/topics/{topic}Get the last received message for a specific MQTT topic

Models

Event {
event_timestring
Time the event occurred
motion_statestring
example: detected
State of motion
seqinteger
Event sequence number
}

EmptiedPayload {
emptied_bystring
example: maintenance-team-A
who emptied the bin
emptied_atstring
example: 2026-05-26T14:00:00Z
ISO timestamp (optional)
}

Σχήμα 4: Η διαδραστική διεπαφή του Swagger UI. Διακρίνονται τα τρία βασικά namespaces του συστήματος: η διαχείριση κάδων (Bins) με ενσωματωμένη παρακολούθηση διαθεσιμότητας και συντεταγμένων, η ανάκτηση στατικών χαρακτηριστικών υλικού (Sensors) και η άμεση παρακολούθηση της κατάστασης του MQTT Broker (MQTT).

2.6 Visualization: Home Assistant Dashboard

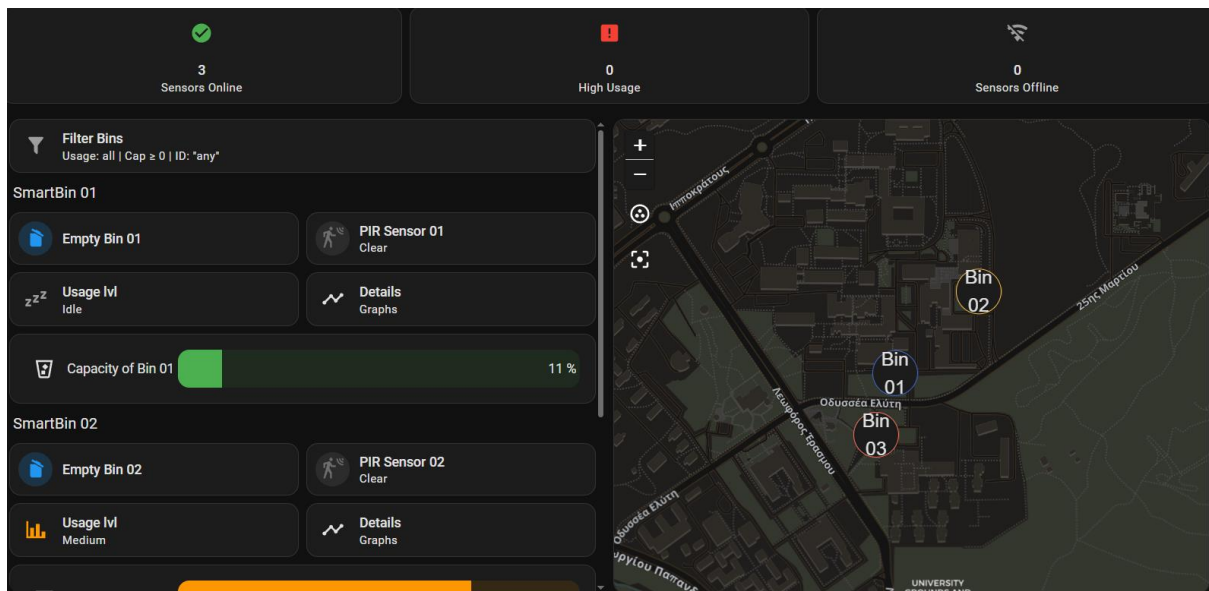
Το τελικό επίπεδο της αρχιτεκτονικής αφορά την οπτικοποίηση των δεδομένων και τη διαδραστική διαχείριση της εγκατάστασης. Επιλέχθηκε η βιομηχανικά καθιερωμένη πλατφόρμα ανοιχτού κώδικα Home Assistant, αξιοποιώντας το πρωτόκολλο MQTT Auto-Discovery.

Μόλις ένας νέος έξυπνος κάδος συνδεθεί στο δίκτυο, τα scripts διαχείρισης (producer.py και analyzer.py) αποστέλλουν αυτόματα δομημένα JSON payloads στα ειδικά topics (homeassistant/.../config). Ως αποτέλεσμα, η πλατφόρμα εγγράφει άμεσα και δυναμικά τις εξής οντότητες για κάθε κάδο, χωρίς καμία χειροκίνητη παρέμβαση:

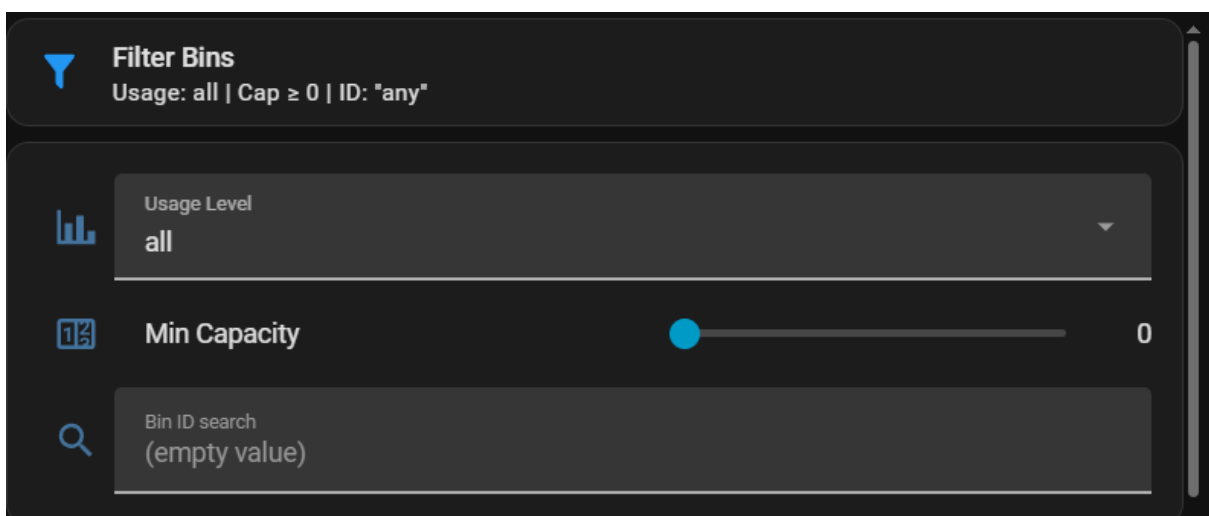
1. **Binary Sensor:** Απεικονίζει την τρέχουσα κατάσταση κίνησης (Detected/Clear).
2. **Sensor (Capacity & Usage):** Καταγράφει τον συνολικό αριθμό ρίψεων (αξιοποιώντας το seq payload).
3. **Button (Command Interface):** Παρέχει έναν διαδραστικό διακόπτη (Empty Bin), ο οποίος, όταν πατηθεί από τον διαχειριστή, στέλνει εντολή ελέγχου στο κανάλι /cleared του κάδου, μηδενίζοντας φυσικά τον μετρητή.

Για την αποδοτική παρακολούθηση του στόλου κάδων, σχεδιάστηκε ένα εξαιρετικά παραμετροποιήσιμο κέντρο ελέγχου (main_dashboard.yaml):

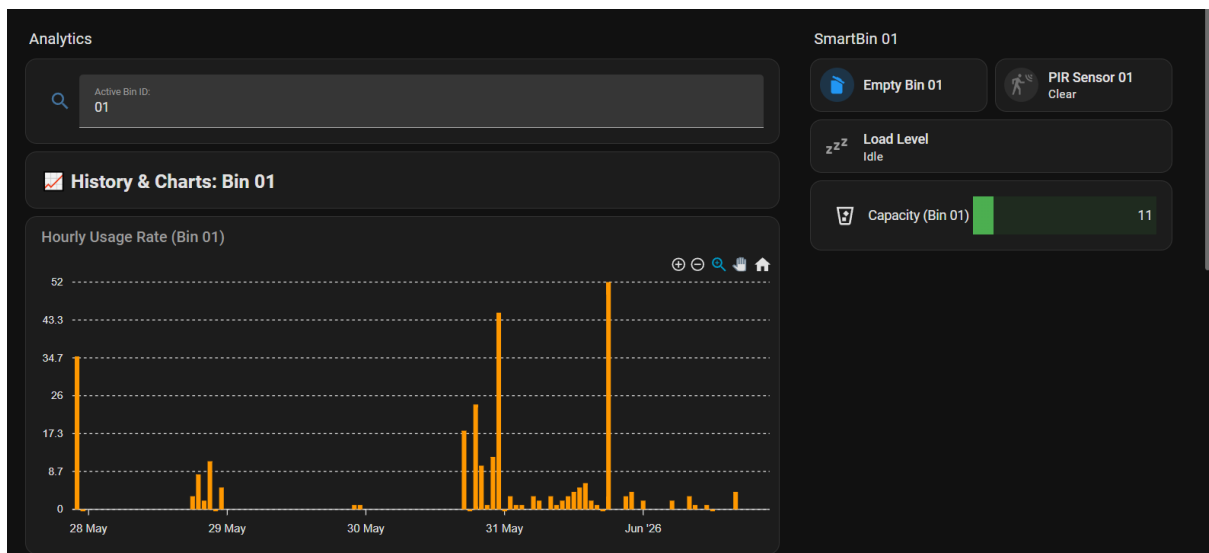
- **Dynamic Filtering:** Μέσω βοηθητικών μεταβλητών (input helpers) που έχουν οριστεί στο helpers.yaml, ο διαχειριστής μπορεί να φιλτράρει δυναμικά την προβολή των κάδων.
- **Geolocation:** Αξιοποιώντας το topic των συντεταγμένων (/coordinates), οι κάδοι τοποθετούνται δυναμικά στον ενσωματωμένο χάρτη της πλατφόρμας.
- **Advanced Analytics:** Έγινε χρήση ApexCharts για την απεικόνιση των χρονοσειρών, προβάλλοντας με ακρίβεια τον ωριαίο ρυθμό χρήσης.



Σχήμα 5: Main Dashboard της εφαρμογής.



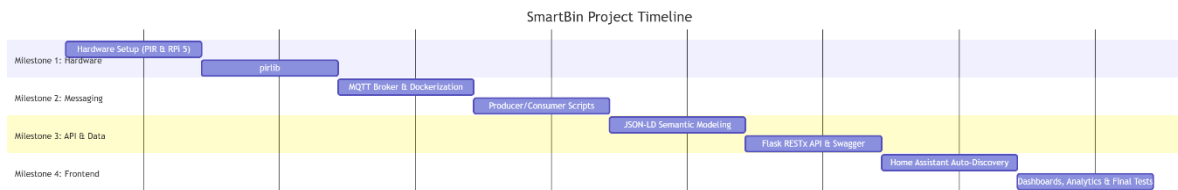
Σχήμα 6: SmartBin Filter.



Σχήμα 7: Main Dashboard της εφαρμογής.

3. Milestones

Η ανάπτυξη του συστήματος ακολούθησε μια ευέλικτη μεθοδολογία, καταναμημένη σε τέσσερα κύρια milestones, καθένα αφιερωμένο σε ένα διακριτό επίπεδο της αρχιτεκτονικής.



Σχήμα 8: Χρονοδιάγραμμα ανάπτυξης του συστήματος (Gantt Chart) καταναμημένο σε 4 κύριες φάσεις, από το επίπεδο του υλικού (Hardware) έως τη διεπαφή χρήστη (UI).

Αναλυτικότερα, η υλοποίηση διαμορφώθηκε ως εξής:

- **Milestone 1: Edge Hardware.** Διασύνδεση του αισθητήρα PIR (HC-SR501) με το Raspberry Pi 5. Παράλληλα, αναπτύχθηκε η βιβλιοθήκη pirlib για την αξιόπιστη δειγματοληψία του φυσικού σήματος.
- **Milestone 2: Messaging & Docker.** Εγκατάσταση του Mosquitto MQTT Broker και ανάπτυξη των μικροπηρεσιών (Producer, Consumer, Analyzer) σε Python. Όλη η στοίβα πακεταρίστηκε σε Docker containers, εξασφαλίζοντας φορητότητα και δυνατότητα δοκιμών κλιμάκωσης με εικονικούς κάδους.
- **Milestone 3: JSON-LD & REST API.** Μετάβαση σε σημασιολογική αρχιτεκτονική (Linked Data) μέσω οντολογιών (BOT, SAREF, SOSA). Ακολούθησε η ανάπτυξη του Flask-RESTx API και η αυτόματη τεκμηρίωσή του μέσω Swagger UI.
- **Milestone 4: User Interface.** Ρύθμιση του Home Assistant μέσω MQTT Auto-Discovery. Υλοποιήθηκαν τα δυναμικά Dashboards, τα γραφήματα χρονοσειρών (ApexCharts) και η διεπαφή αμφίδρομου ελέγχου (κουμπί Empty Bin) για την τελική διαχείριση της εγκατάστασης.

4. Συμπεράσματα

Η παρούσα εργασία παρουσίασε την ανάπτυξη ενός συστήματος SmartBin, αναδεικνύοντας τα πλεονεκτήματα μιας event-driven αρχιτεκτονικής. Τα βασικά τεχνικά επιτεύγματα της υλοποίησης συνοψίζονται στα εξής:

- **Scalability:** Μέσω της χρήσης Docker και μικροπηρεσιών, το σύστημα κλιμακώνεται, επιτρέποντας την αξιόπιστη προσομοίωση εικονικών κάδων στο ίδιο δίκτυο, χωρίς επιπλέον απαιτήσεις σε φυσικό υλικό.
- **Σημασιολογική Διαλειτουργικότητα:** Η χρήση του προτύπου JSON-LD και παγκόσμιων οντολογιών (BOT, SAREF, SOSA) διασφαλίζει ότι τα παραγόμενα γεγονότα είναι μηχανικά αναγνώσιμα, έτοιμα για άμεση ενσωμάτωση σε smart οικοσυστήματα.
- **Αξιοπιστία από το Edge έως το UI:** Η ανάπτυξη προσαρμοσμένου λογισμικού στο υλικό (pirlib) εξασφάλισε ακριβή τηλεμετρία, ενώ το πρωτόκολλο MQTT προσέφερε ταχύτατο συγχρονισμό μεταξύ του Edge Layer και του Application Layer (Flask API & Home Assistant).

Η αρχιτεκτονική μπορεί να εξελιχθεί περαιτέρω με την ενσωμάτωση μιας για την αποθήκευση του τεράστιου όγκου των events και αυτό θα αποτελέσει το θεμέλιο για την εκπαίδευση μοντέλων machine learning. Έτσι, το σύστημα θα μεταβεί από την απλή παρακολούθηση, στην δυναμική βελτιστοποίηση των δρομολογίων αποκομιδής κάδων.